

Introduction

Background: Borrowing and Lending On Chain

By default, money sits in wallets. It does nothing.

A holder of 10,000 USDC faces a choice: leave it idle, or put it to work. Traditional finance offers savings accounts, money market funds, certificates of deposit. On chain, the equivalent is a lending pool. The holder deposits USDC into a smart contract. That contract makes the USDC available for others to borrow. In return, the depositor earns interest.

This is the lender's side.

On the other side sits someone who already owns an asset, say SOL, and wants dollars without selling. Perhaps they believe SOL will appreciate and refuse to part with it. Perhaps they need liquidity for an opportunity but cannot access traditional credit. Perhaps they operate a trading strategy that requires both the upside exposure and the working capital.

They deposit SOL into the same smart contract as collateral. The contract allows them to borrow USDC against that collateral, up to a ratio determined by the protocol's risk parameters. If they deposit 150 SOL worth \$15,000, and the loan-to-value ratio is 66%, they can borrow up to \$10,000 USDC.

This is the borrower's side.

The smart contract holds both: the lender's USDC, the borrower's SOL. The borrower takes USDC out and pays interest on it. That interest flows to the lender. The collateral remains locked until the debt is repaid. If the value of the collateral drops too far relative to the debt, liquidators can seize and sell the collateral to repay lenders, protecting them from borrower default.

Two strangers, no intermediary. The collateral is the underwriting.

Now scale this. The smart contract does not match one lender to one borrower. It aggregates. A thousand lenders deposit into the same pool. A hundred borrowers draw from it. The design that worked for two participants works for two thousand. The pool holds all deposited capital, tracks all collateral, enforces all liquidations. It scales without adding operational complexity.

This is pool-based lending. Each lender earns a share of all interest paid by all borrowers. Each borrower pays into a common interest stream distributed to all lenders. The pool is the

intermediary, but it is not a bank. It is code. It cannot make discretionary decisions. It follows rules.

A critical rule is how the pool determines interest rates.

The Utilisation Curve

A lending pool contains some amount of deposited capital. Some of that capital is borrowed, some sits idle. The ratio of borrowed capital to total deposited capital is called utilisation.

None

```
utilisation = borrowed_amount / deposited_amount
```

A pool with \$100M deposited and \$70M borrowed has 70% utilisation.

The interest rate borrowers pay is determined by a curve that maps utilisation to rate. At low utilisation, rates are low. At high utilisation, rates are high.

The logic is simple. When utilisation is low, capital is abundant and borrowers are scarce. The protocol wants to attract borrowers, so it offers cheap rates. When utilisation is high, borrowers are abundant and capital is scarce. The protocol wants to attract lenders, so it raises rates. Lenders earn more, which draws deposits. Borrowers pay more, which discourages borrowing. The system self-corrects toward equilibrium.

A typical curve might look like this:

Utilisation	Borrow Rate
0%	0%
50%	4%
80%	8%
90%	20%
95%	50%
100%	70%

The curve is usually gentle at low utilisation and steep at high utilisation. The steepness near 100% is deliberate. It creates strong pressure: borrowers face punishing rates, lenders earn exceptional yields, and both forces push utilisation back down before the pool runs dry.

Lenders do not earn the borrow rate directly. They earn the borrow rate multiplied by utilisation. If borrowers pay 8% and utilisation is 70%, lenders earn approximately 5.6%. The idle 30% dilutes returns.

Why People Borrow

Four primary motivations drive on-chain borrowing.

Leverage. A trader believes SOL will rise 30% over the next month. They hold \$100,000 of SOL. Instead of buying more SOL with fresh capital, they deposit their SOL as collateral, borrow \$50,000 USDC, and use that USDC to buy more SOL. Now they have exposure to \$150,000 of SOL. If SOL rises 30%, their \$150,000 becomes \$195,000. After repaying the \$50,000 loan plus interest, their profit is roughly \$45,000 minus interest, instead of \$30,000 from an unleveraged position.

The risk inverts if SOL falls.

Carry trades (looping). A sophisticated participant notices that a yield-bearing token pays 8% APY while borrowing USDC costs 4%. They deposit collateral, borrow USDC at 4%, and deploy that USDC into the yield-bearing token at 8%. The spread, roughly 4%, is their profit. They can loop this multiple times, each iteration adding leverage and amplifying the spread.

The entire trade collapses if borrow rates spike above the yield. A 4% spread becomes a 4% loss overnight.

Liquidity without sale. A founder holds tokens in their own protocol. Selling would signal a lack of confidence. But they need dollars for operating expenses, personal liquidity, or investment opportunities. Borrowing against their tokens provides liquidity without triggering a taxable event, without reducing their stake, and without sending a negative signal to the market.

The tokens remain theirs. The debt remains their obligation.

Broker financing. A broker offers clients a service: deposit SOL, receive USDC at 7% interest. The client wants simplicity, custody, and a fixed rate they can understand. The broker provides this, but the broker does not hold idle USDC to fund every client loan. Instead, the broker deposits the client's SOL as collateral and borrows USDC from on-chain pools, refinancing at whatever rate the pool offers.

If the pool charges 5%, the broker pockets 2% spread. If the pool charges 8%, the broker loses 1% on every dollar lent. The broker's entire margin depends on borrowing cheaper than the rate promised to clients. Rate volatility is existential.

Why People Lend

Two primary motivations drive on-chain lending.

Yield on idle capital. Stablecoins do not appreciate. A holder of USDC who simply holds earns nothing. Lending provides a return on otherwise dormant capital. The yield fluctuates with market demand for borrowing, but it is strictly better than zero.

Risk-adjusted returns. Collateralised lending is safer than unsecured lending. The borrower cannot simply walk away. If they default, the collateral covers the debt. A lender accepting 6% on a fully collateralised loan faces less risk than a lender accepting 10% on an unsecured loan elsewhere. For capital that prioritises preservation over growth, the risk-adjusted return matters more than the nominal return.

Utilisation Based Rates: The Good, The Bad, and The Ugly

The Good: A Self-Correcting Feedback Loop

The utilisation curve is an elegant mechanism. It encodes a feedback loop that keeps the pool healthy without human intervention.

When borrowing demand surges, utilisation rises, rates rise with it. Higher rates do two things simultaneously: they attract new lenders seeking better yields, and they discourage marginal borrowers unwilling to pay elevated costs. Both forces push utilisation back down.

When borrowing demand falls, utilisation drops, rates drop with it. Lower rates do two things simultaneously: they discourage lenders seeking better yields elsewhere, and they attract new borrowers who find cheap capital appealing. Both forces push utilisation back up.

The system expands and contracts around an equilibrium without anyone coordinating it.

The curve also optimises for instant liquidity. Notice the steepness near 100% utilisation. A pool at 95% utilisation might charge 50% APR. At 99%, perhaps 65%. At 100%, the maximum. This progression exists for a reason: the pool never wants to hit 100%. At 100% utilisation, lenders cannot withdraw. Their capital is fully borrowed. The steep curve creates pressure to maintain a buffer of idle capital, ensuring lenders can exit at will.

A simple mathematical function, applied to a single ratio, produces a self-regulating market that balances supply, demand, and liquidity simultaneously.

The Bad: Volatility as a Feature

The same responsiveness that makes the system self-correcting makes it volatile.

A single large borrower enters, takes \$30M from a \$100M pool, and utilisation jumps from 50% to 80%. Rates double. Every existing borrower now pays more. A single large lender exits, withdraws \$20M from the same pool, and utilisation jumps from 80% to near 100%. Rates explode. Existing borrowers face punishing costs.

Neither side controls the rate they will pay or receive tomorrow.

A borrower who enters at 5% might pay 15% next week. A lender who enters at 7% might earn 2% next week. Both committed capital based on conditions that no longer exist.

The Bad: No Price Discovery

Price discovery requires two parties expressing what they want and finding where their preferences meet. A buyer says "I will pay X." A seller says "I will accept Y." The market clears where X and Y overlap. Both sides have agency. Both sides signal intent.

The utilisation curve offers no such mechanism. The rate is a function of a ratio. Borrowers and lenders do not state their desired rate. They observe the current rate and decide to participate or not. They are rate takers, not rate makers.

A borrower who would pay 6% for certainty cannot say so. A lender who would accept 5% for size cannot advertise it. The curve does not know their preferences exist. It only knows the current utilisation and outputs a rate from the formula.

The feedback loop adjusts rates based on aggregate behaviour, but this is not price discovery. It is price setting by algorithm. The rate moves when people act, but no one can express terms before acting. There is no bid. There is no ask. There is only the curve.

Price discovery emerges from visible, competing offers. The utilisation curve produces a single rate that everyone must accept or reject.

The Ugly: Predictability is Impossible

This volatility is not a bug. It is the mechanism by which pools balance supply and demand. But it makes certain activities impossible.

A loopster cannot commit to a carry trade if the borrow rate might exceed the yield within days. An institution cannot budget financing costs if those costs are unforecastable. A broker cannot promise clients a rate if their own refinancing cost is unknowable. A lender cannot make commitments to their own investors if their yield might collapse.

The consequences are visible. Carry trades have blown up. Positions that looked profitable at inception inverted within weeks, leaving participants with losses they could not have modelled. Institutions that might deploy serious capital into DeFi lending look at the rate volatility and decline. Their risk frameworks require forecastable costs. Variable rates offer no such thing.

The result is shallow pools. Capital that would participate stays on the sidelines. Borrowers who need size cannot find it. Lenders who want duration cannot get it.

The pool solves for liquidity via self-correction and trades off predictability for it.

This was acceptable when DeFi lending served a narrow set of participants comfortable with volatility. It is no longer acceptable. Institutions, fintechs, and exchanges are moving on chain. They bring larger capital, longer time horizons, and stricter requirements. Variable rate borrowing has hit a ceiling. The primitive needs to evolve.

Kamino Fixed Rates: A new primitive

Kamino is launching Fixed Rates to finally solve interest rate uncertainty and interest rate price discovery in DeFi.

The (variable) status quo

Virtually all of DeFi borrowing is currently happening on variable rates. Variable rates create problems for the two main use cases of borrowing in DeFi: looping and size borrowing.

Consider a loopster running a simple carry trade: borrow USDC at 4%, earn 6% on a yield bearing token, pocket the spread. The position works until borrow rates spike to 9% and the entire trade inverts. Or take an institution that borrows \$20M at 5% to finance a client willing to pay 6%, pocketing 1% spread. Two weeks later utilisation moves, the rate jumps to 12%, and the position becomes uneconomical overnight. Both cases need rate stability and neither can get it.

Lenders face the inverse problem.

A lender deploys \$50M into a pool yielding 7%. Two things can happen: the size is too large relative to the pool and yields dilute to 3%. Or a large borrower repays and

suddenly the rate is 3%. The yield collapsed. There was no warning, no ability to lock the better rate when it existed.

In simple terms: both borrowers and lenders are subject to an ever-changing interest rate curve and points on that curve. They deposit or borrow given what the pool offers, or they leave. Precisely, they are rate takers.

The structural problem is that neither side can fully express their intent. A lender willing to pay 5.5% for three months of certainty has no way to broadcast that offer. An institution needing \$20M at 6% for six months cannot signal that demand. A vault managing \$50M that would lock in 5.5% for three months has no way to advertise it. When someone needs size, they show up and take whatever liquidity exists at whatever rate the utilisation curve happens to show. If that pushes rates up, new lenders notice and deposit (if at all). But the sequence is backwards. The borrower already paid elevated rates just to signal demand. The lender maybe arrived, later, after the fact with no guarantee rates stay high. Nobody stated their actual terms. The market just reacted.

Intents, fixed rates and fixed terms

Fixed Rates lets both sides (1) communicate what they want before anything commits, as well as (2) allowing those terms to happen - rates to be fixed for the specified period.

Briefly, how it works: borrowers post (Borrow) Orders on a grid of curves differentiated by rate and duration. Simple: "I want to borrow \$10M USDC at 5.5% for 3 months, collateralised with SOL, in the main market." Vaults place Conditional Liquidity on the same grid: "Would lend at 5.5% for 3 months in the main market and at 7% in the Bitcoin market." Critically, they do not incur opportunity costs for expressing that intent: that vault's money stays fully deployed in variable-rate curves earning whatever the current rate is. It's not sitting idle waiting for a match. When a Borrow Order hits and terms align, the money moves atomically from the variable pool into the fixed-rate loan. (Previous functionality remains: an atomic order placing and filling becomes a taking of liquidity - the borrower doesn't have to wait if conditional liquidity already exists at desired terms).

Now the borrower has locked 5.5% for the full three months. The lender has locked 5.5% for the full three months. The lender can actually plan the carry trade without borrow rate spike risk. The institution can budget around stable financing costs. The lender committed to a specific yield for a specific duration and won't watch it collapse.

Price discovery

The grid makes the liquidity visible without committing capital - a borrower can see that vaults are offering \$30M at 6% for three months but only \$5M at 5%. Equally, all vaults can see that

borrowers want \$40M at 5.5% for six months. Both can adjust their orders to take from the other side, or keep their intents as they are. That is basically price discovery.

An Emerging Yield Curve

Traditional finance has a yield curve. It shows the cost of borrowing across time. One month Treasury bills pay 4.3%. Three month bills pay 4.5%. Ten year bonds pay 5.1%. The reason is that locking money for longer should be compensated, because more can go wrong in more time. Duration carries risk. Risk carries premium.

DeFi has never had this.

Variable rate pools collapse all duration into a single point. A borrower at 5% today might pay 12% tomorrow. There is no term structure because there are no terms.

The fixed rate grid changes this. Place the reserves side by side. USDC at 4.5% for one month. USDC at 5.0% for three months. USDC at 5.5% for six months. The structure reveals what the market charges for time.

Term Premium

The fixed rate grid introduces duration. Rates across different terms, visible simultaneously, reveal the term premium: the additional yield lenders require for committing capital longer.

If one month USDC trades at 4.5% and three month trades at 5.0%, the term premium for that additional duration is 50 basis points.

The grid will be short at first. One month, three months, perhaps six. Yields across tenors may differ only slightly. But the infrastructure now exists. Borrow Orders and Conditional Liquidity give both sides a language to express and commit to terms across time. That language did not exist before.

How it develops remains to be seen. The mechanism now exists. The premium will be determined by supply and demand: Borrow Orders on one side, Conditional Liquidity on the other, rates settling where the two intersect.

Technical Specification

Overview

Fixed Rates introduces interest rate certainty and price discovery to Kamino through a system of **multiple reserves** with **fixed rates** and **fixed terms**. Instead of a single USDC reserve with variable rates determined by utilisation, markets now also support multiple USDC reserves, each with a specific fixed rate and duration (e.g., USDC 5.5% 3m, USDC 6.0% 6m). Borrowers lock financing costs for the **full term**. Lenders lock yields for the full term. Neither side faces rate volatility after commitment. The system enables price discovery through **Borrow Orders** (borrowers stating desired terms) and **Conditional Liquidity** (vaults signalling lending availability while keeping capital deployed in variable pools). The standardised grid of rate-duration pairs creates a yield curve showing market prices across the term structure.

Main Components

1. **Markets and Reserves (Curves) Markets:** Isolated lending environments with specific collateral and debt token combinations. Each market operates independently with its own risk parameters and reserves. **Reserves:** Individual lending pools within a market. Each reserve is either variable-rate or fixed-rate with a specific term. Multiple reserves for the same debt token (eg USDC) can coexist in one market, with its own liquidity, borrowed amounts, and maturity schedule. Previously there was one USDC reserve with upward sloping interest rate curve and rate determined by utilisation. Instead, for flat curves, the interest rate borrowers pay is always the same.
2. **Vaults:** Abstraction layer that aggregates lender capital and manages allocation across multiple reserves and markets. Single-token pools (e.g., USDC vault) that handle complexity of reserve selection, liquidity management, and maturity matching on behalf of depositors.
3. **Conditional Liquidity:** Vault-placed signals of lending availability on specific reserves. Capital remains in variable-rate pools earning baseline yields until Borrow Orders activate atomic transfers into fixed-rate reserves.
4. **Borrow Orders:** Borrower requests for loans at specific terms. Orders serve two roles: (1) trigger atomic movement of Conditional Liquidity from variable pools into fixed-rate reserves when matches occur and (2) stay open allowing for vault managers or liquidity providers to see where there is borrow demand and later fill them.
5. **Withdrawal Queue:** FIFO queue allowing lenders to signal exit intent. Queued liquidity blocks rollover, forcing borrower repayment and enabling lender exit at term end.
6. **Rollover Mechanism:** At loan maturity, borrowers can extend loans for another term period if sufficient liquidity is available and not queued for withdrawal.

1. Markets and Reserves

Market Structure - A **market** is an isolated lending environment with:

- Specific collateral tokens allowed
- Specific debt tokens available for borrowing

- Independent risk parameters (LTV ratios, liquidation thresholds, rehypothecation)
- Multiple reserves for each debt token

Example: Main Market

Collateral reserves: SOL, BTC, ETH

Debt reserves:

- USDC variable-rate reserve (0-70% APR based on utilization)
- USDC 5.0% 3m fixed-rate reserve
- USDC 5.5% 3m fixed-rate reserve
- USDC 6.0% 3m fixed-rate reserve
- USDC 6.5% 3m fixed-rate reserve

Example: A borrower deposits SOL as collateral and can borrow USDC from any of the USDC reserves (variable or any fixed-rate reserve).

Market Isolation - Markets are completely isolated:

- Collateral in Main Market cannot be used to borrow in JLP Market
- Liquidity in Main Market USDC 5.5% 3m is separate from JLP Market USDC 5.5% 3m
- Same rate-duration pair creates different reserves in different markets

Why isolation matters - Risk parameters differ by collateral type. JLP might require different LTV ratios than SOL. Isolating markets prevents risk contagion between collateral types.

NEW Reserve Types

Within each market, debt tokens have multiple reserves:

1. **(Traditional) Variable-Rate Utilisation model** - previously, each market had one reserve per debt token with utilisation-based interest rates.

Rust

```
let utilisation = borrowed_amount / deposited_amount;
let borrow_rate = borrow_curve.interpolate(utilisation);
```

Example: One USDC reserve with rate formula. As utilisation increases from 0% to 100%, rate increases from base rate (e.g., 0%) to a typical maximum rate (e.g.,

70%). On a single reserve, rate changes continuously based on borrowing and lending activity.

1. **Fixed-Rate Model** - in this model, the borrower pays the same rate, regardless of utilisation. The properties of a fixed rate reserve are:
 - **Fixed Rate:** Interest rate is constant regardless of utilisation. USDC 5.5% 3m charges each borrower 5.5% whether utilisation is 10% or 90%. (Lenders obviously get less if utilisation is below 100%, but the system optimises for 100% utilisation, as we will see later).
 - **Fixed Term:** Each reserve has a duration parameter. When a loan originates, it has a maturity time of `loan_start_time + duration`.
 - Optional: Early cancellation fee on the first term (not on subsequent roll overs)

(More on fixed rate maturity and liquidations later).

NEW The resulting debt reserve grid

An example set of all available reserves forms a grid organised by rate and duration:

Rust

USDC Reserves on Main Market:

	1m	3m	6m	12m
4.5%	•	•	•	•
5.0%	•	•	•	•
5.5%	•	•	•	•
6.0%	•	•	•	•
0-70% (variable, no maturity)				

All reserves within a market are shared by all collaterals. Custom curves (reserves) and collaterals can also be defined pairwise within elevation groups, for example BTC-USDC 12m 6%, this comes out of the box.

Borrowing Flow

Step 1: Choose Market Borrower selects market based on collateral they have (SOL → Main Market, JLP → JLP Market)

Step 2: Deposit Collateral Collateral goes into market's collateral reserve

Step 3: Choose Debt Reserve Borrower selects which reserve to borrow from:

- Variable-rate for flexible terms
- Fixed-rate for rate certainty

Step 4: Borrow

- If `available_liquidity > 0`: direct borrow (standard flow)
- If `available_liquidity = 0`: place Borrow Order (explained in section 4)

Cross-Reserve Borrowing (Same Market)

A borrower with collateral in one market can have multiple loans from different reserves in that market simultaneously.

Example:

- Borrower has 100 SOL deposited in Main Market
- Borrows 10K USDC from Main Market USDC variable reserve
- Also borrows 20K USDC from Main Market USDC 5.5% 3m reserve
- Also borrows 15K USDC from Main Market USDC 6.0% 3m reserve

2. Vaults

Current State

Vaults are an established abstraction layer on Kamino that aggregate lender capital and manage allocation across reserves. They are heavily used and critical infrastructure.

A vault is a single-token pool (e.g., USDC vault) that:

- Accepts deposits from lenders
- Issues vault shares representing proportional ownership
- Manages allocation across multiple reserves and markets
- Handles rebalancing based on risk manager decisions

Why Vaults Exist

Without vaults, lenders would need to:

- Manually select which reserves to deposit into across multiple markets
- Monitor yields across dozens of reserves
- Move capital between reserves as conditions change
- Handle complexity of multiple positions
- Active management > passive management.

Vaults professionalize this. Lenders deposit once. Vault managers (risk managers) handle all allocation decisions.

Vault Structure

Example: USDC Prime Vault

Lenders deposit USDC. Vault manager allocates:

- 60% to Main Market USDC variable reserve
- 30% to JLP Market USDC variable reserve
- 10% to Altcoin Market USDC variable reserve

Allocations are committed. Capital physically moves into these reserves. Earns whatever those reserves currently yield.

Photo 1: USDC Prime Vault allocations

Photo 2: USDC Prime Vault and USDC Frontier Vault allocations

Allocation Management

Vault managers actively adjust allocations based on:

- Relative yields across markets
- Risk assessment of different collateral types
- Utilisation levels in different reserves
- Overall market conditions

3. Conditional Liquidity NEW

Conditional Liquidity allows vaults to signal availability on fixed-rate reserves while keeping capital deployed in variable-rate reserves earning yields. Capital only commits to fixed rates when actual borrower demand (via Borrow Orders) materialises.

The Problem Without Conditional Liquidity

If vaults wanted to provide liquidity to fixed-rate reserves directly, they would face two problems:

1. **Opportunity cost:** Capital sitting in a fixed-rate reserve with no borrowers earns nothing. A vault allocating 20% to a 5.5% 3m reserve would have that capital idle until someone borrows.

2. **Liquidity fragmentation:** Capital split across many fixed-rate reserves (different rates, different durations, different markets) means small amounts in each reserve, making it difficult for large borrowers to find sufficient liquidity at any single rate-duration pair.

How Conditional Liquidity Works

Vaults specify a percentage of their total capital as conditionally available to specific fixed-rate reserves. This capital remains physically deployed in variable-rate reserves, earning whatever those reserves currently yield.

When a Borrow Order matches the conditional terms, the system atomically:

1. Withdraws the required amount from variable-rate reserves
2. Deposits it into the matched fixed-rate reserve
3. Executes the loan to the borrower

The vault's actual allocation only changes at the moment of match.

Multi-Reserve Placement

A vault can place Conditional Liquidity on multiple reserves simultaneously using the same underlying capital. For example, a vault with \$100M deployed in variable reserves could signal:

- \$50M conditionally available at 5.0% 3m
- \$50M conditionally available at 5.5% 3m
- \$30M conditionally available at 6.0% 6m

These overlap intentionally. The \$100M is not reserved or partitioned - it remains fully deployed in variable reserves. Whichever Borrow Order arrives first and matches any of these conditions triggers the atomic transfer. Once \$50M moves to fill a 5.0% 3m order, the remaining \$50M in variable reserves can still fill other conditional placements up to that remaining amount.

Why Vaults make Fixed Rate viable

- Maturity mismatch - Some lenders want instant liquidity. Fixed-rate loans lock capital for 3-6 months. Nobody wants to be stuck for 6 months. But having a vault that abstracts away and only allocates a portion of liquidity to fixed rates and the rest to variable rates, gives a buffer to the lenders that want to get out fast. Pooling liquidity allows for better liquidity and maturity management and buffer.
- Complexity abstraction - allocating to fixed rates across terms is a complex decision. Most people are not equipped to do so and monitor it. But since Vaults are a new abstraction layer, the managers can absorb this new complexity.

No Fill Priority

When multiple vaults have Conditional Liquidity on the same reserve and a Borrow Order arrives, there is no priority ordering. The first vault whose `invest_to_fill_borrow_order()` instruction executes successfully fills the order. This is determined by transaction ordering, not by any queue or preference system.

Modifying Conditional Liquidity

Conditional Liquidity is not a discrete order that gets placed and cancelled. It is a configuration parameter on the vault—specifically, the percentage allocated to each conditional reserve. Vault managers adjust Conditional Liquidity by changing these allocation percentages. Setting a conditional allocation to 0% removes the signal entirely.

Why Vaults Enable Fixed-Rate Lending

Vaults solve two structural problems that would otherwise make fixed-rate lending impractical for individual lenders:

1. **Maturity mismatch:** Individual lenders often want liquidity on demand. Fixed-rate loans lock capital for months. A vault that allocates only a portion of its capital to fixed rates while keeping the rest in variable reserves creates a liquidity buffer. Lenders can withdraw from the vault (subject to available liquidity), even while some vault capital is locked in fixed-rate positions.
2. **Complexity abstraction:** Deciding how to allocate across multiple fixed-rate reserves with different terms, rates, and markets is operationally complex. Vault managers absorb this complexity on behalf of depositors.

Why Fixed Rates Enable Higher Vault Yields

Fixed-rate reserves tend toward 100% utilisation because Conditional Liquidity only commits when matched. Unlike variable reserves where idle liquidity dilutes returns, fixed-rate reserves have capital deployed only when borrowed. A 6% fixed-rate reserve at 100% utilisation pays lenders 6%, not the diluted rate that variable reserves with idle liquidity would produce.

Additionally, longer lock-up periods command higher rates. The term structure creates yield premium opportunities that vault managers can capture through strategic allocation.

Example vault allocation

Step 1:

- \$80m in Main Market (Variable Rate)
- \$20m in JLP Market (Variable Rate)
- **Conditional: \$10m in Main Market 5%, 3 months**

Step 2 - Atomically: new borrower places Borrow Order for \$10 at 5%, 3m, liquidity gets moved from one (any) reserve

- \$70m in Main Market (Variable Rate)
- \$20m in JLP Market (Variable Rate)
- \$10m in Main Market (5%, 3m)

Step 3 - Rebalancing (to maintain the proportions)

- \$72m in Main Market (Variable Rate)
- \$18m in JLP Market (Variable Rate)
- \$10m in Main Market (5%, 3m)

4. Borrow Orders **NEW**

Borrow Orders allow borrowers to request loans at specific terms. They serve two functions:

1. **Trigger mechanism:** Activate atomic transfer of Conditional Liquidity from variable reserves into fixed-rate reserves when terms match.
2. **Demand signalling:** Express desired terms when insufficient liquidity currently exists, remaining visible for vault managers to discover and potentially fill later.

Order Structure

A Borrow Order specifies:

Parameter	Description
Obligation	The borrower's existing loan account with pre-deposited collateral in a specific market
Borrow Token	The debt token requested (e.g., USDC)
Min Borrow Duration	Minimum acceptable term (e.g., 1m, 3m)
Max Borrow Rate	Maximum acceptable interest rate (e.g., 6%)
Borrow Amount	Quantity of debt tokens requested

The borrower must have sufficient collateral already deposited in the target market. The order does not post new collateral - it borrows against existing collateral.

Order Lifecycle

- **Placement:** Borrower creates order specifying terms. Order becomes visible on-chain.
- **Persistence:** Orders do not expire. They remain active until explicitly cancelled by the borrower or fully filled.
- **Modification:** Borrowers can modify order parameters (amount, rate, duration) without cancelling and recreating.
- **Cancellation:** Borrowers can cancel unfilled or partially filled orders at any time.

Fill Mechanics

Atomic Fills (Conditional Liquidity exists):

When Conditional Liquidity exists at terms matching the Borrow Order, the fill executes atomically:

1. Vault calls `invest_to_fill_borrow_order()` instruction
2. This calls `fill_borrow_order()` on the lending market
3. Health checks verify post-borrow solvency (LTV within limits given collateral and new debt)
4. If checks pass: liquidity moves from variable reserve to fixed-rate reserve, debt originates
5. If checks fail: transaction reverts, no state changes

For borrowers wanting instant execution, the entire flow can occur in a single transaction:

- KLen: Init Obligation → Deposit Collateral → Place Borrow Order
- KVault: `invest_to_fill_borrow_order()` (calls `fill_borrow_order()`)

This atomic Place + Fill pattern functions as a `market order` - taking available liquidity immediately.

Partial Fills:

If Conditional Liquidity exists but is insufficient to fill the entire order:

- Available amount fills immediately
- Order remains active for the unfilled remainder
- Subsequent Conditional Liquidity or direct deposits can fill the remainder

Example: Borrower places order for \$10M at 5.5% 3m. Only \$4M Conditional Liquidity available. \$4M fills immediately, order remains open for \$6M.

Delayed Fills (No Conditional Liquidity):

If no matching Conditional Liquidity exists:

- Order remains open and visible on-chain
- Vault managers can discover unfilled orders and see aggregate demand at various rate-duration pairs
- Vaults can later call `fill_borrow_order()` directly or adjust Conditional Liquidity allocations to match
- Individual lenders can also fill orders by depositing directly to the relevant reserve and calling `fill_borrow_order()`

No Fill Priority

When multiple Borrow Orders exist on the same reserve and liquidity becomes available, there is no priority ordering between orders. Whichever `fill_borrow_order()` instruction executes first receives the liquidity. Fill order is determined by transaction ordering, not by submission time or any queue.

5. Withdrawal Queue NEW

Fixed-rate reserves lock liquidity for specific terms. Lenders and vaults accept temporary illiquidity in exchange for rate certainty. Without an exit mechanism (or without the ability to spike rates to force borrowers out), borrowers could roll loans indefinitely, trapping lender capital permanently.

The Withdrawal Queue allows lenders and vaults to signal exit intent. Queued liquidity blocks rollover, forcing borrower repayment at term end and enabling lender exit.

Queue Structure

Each fixed-rate reserve maintains its own independent FIFO (first-in, first-out) queue of withdrawal tickets.

Who can submit tickets: Any entity with capital committed to that specific reserve—vaults or individual lenders who deposited directly.

Ticket parameters: Amount requested for withdrawal.

No notice period: Tickets take effect immediately upon submission. There is no required advance notice before maturity.

Ticket Lifecycle

Submission: Lender or vault submits ticket specifying withdrawal amount. Ticket enters queue, `queued_liquidity` increases.

Cancellation: Tickets can be cancelled at any time before being filled. Cancellation removes the ticket from the queue and decreases `queued_liquidity`.

Filling: Tickets fill when `available_liquidity` in the reserve becomes sufficient.

Queue Processing

The queue processes whenever `available_liquidity` increases in the reserve. Processing is automatic—lenders do not need to manually trigger withdrawal.

Fulfilment triggers (either of):

- Loan repayments to that reserve
- New deposits to that reserve
- Liquidations of loans from that reserve

Processing order: Strictly FIFO. The queue does not advance to the next ticket until the current first ticket is completely filled.

Partial fills: If `available_liquidity` is less than the first ticket's requested amount, the ticket partially fills. The ticket remains first in queue for the remaining amount. No other tickets fill until the first ticket completes.

Example:

- Ticket A: requests \$50M, position 1
- Ticket B: requests \$30M, position 2
- \$20M `available_liquidity` arrives
- Ticket A receives \$20M, now requests \$30M remaining, stays position 1
- Ticket B receives nothing, stays position 2
- Another \$30M arrives
- Ticket A receives \$30M, fully filled, exits queue
- Ticket B moves to position 1

Interaction with Rollover

Queued liquidity directly affects rollover eligibility. The principle is simple: **lenders who want out take priority over borrowers who want to stay in.**

When a loan matures, its principal returns to the reserve as available liquidity. The borrower may want to immediately re-borrow that same capital (rollover). But if lenders have queued withdrawal tickets, their exit claims come first.

Concretely, a loan can only roll over if enough liquidity remains after satisfying all queued withdrawals:

Term	Meaning
<code>borrowed_amount</code>	Principal of the maturing loan that wants to roll over
<code>available_liquidity</code>	Unborrowed capital currently sitting idle in the reserve
<code>queued_liquidity</code>	Total amount lenders have requested to withdraw

Rollover succeeds when: `borrowed_amount <= available_liquidity - queued_liquidity`

In practice, at least one of `available_liquidity` or `queued_liquidity` is always zero. When available liquidity exists and a withdrawal ticket is submitted, the queue immediately processes and the ticket fills—leaving no available liquidity. Conversely, when there's no available liquidity and a ticket is submitted, it waits in the queue. The two values don't coexist because one always consumes the other.

This means the rollover check simplifies to: if any queued liquidity exists, rollover is blocked.

When a loan matures, its principal conceptually returns to `available_liquidity`. If lenders have queued withdrawal tickets, those claims take priority over the borrower's desire to re-borrow.

Example: Queue Blocks Rollover

Reserve: Main Market USDC 5.5% 3m

Day	Event	available_liquidity	queued_liquidity
0	Vault deposits 100M, Borrower borrows 100M	0M	0M
30	Vault submits withdrawal ticket for 100M	0M	100M
90	Loan matures, rollover check: $100M \leq (100M - 100M) \rightarrow \text{FALSE}$	100M	100M
90	Rollover blocked, borrower must repay within grace period	100M	100M
90 + 2h	Borrower repays, queue processes	0M	0M

Vault receives 100M and exits.

Example: New Deposit Fills Queue

Reserve: Main Market USDC 5.5% 3m

Day	Event	available_liquidity	queued_liquidity
0	Vault A deposits 100M, Borrower borrows 100M	0M	0M
5	Vault A submits withdrawal ticket for 50M	0M	50M
6	Vault B deposits 50M directly to reserve	50M	50M
6	Queue processes, Vault A's ticket fills	0M	0M

Vault B's deposit immediately exits to fulfil Vault A's withdrawal. Vault B effectively replaced Vault A as lender to the existing loan.

Example: Liquidation Fills Queue

Reserve: Main Market USDC 5.5% 3m

Day	Event	available_liquidity	queued_liquidity
0	Vault deposits 100M, Borrower borrows 100M (collateralised with SOL)	0M	0M
30	Vault submits withdrawal ticket for 100M	0M	100M
45	SOL price drops, borrower's position becomes undercollateralised	0M	100M
45	Liquidator repays 100M debt, seizes collateral	100M	100M
45	Queue processes, vault's ticket fills	0M	0M

Vault receives 100M and exits. The liquidation event returned debt capital to the reserve, triggering queue processing regardless of whether the loan had reached maturity.

Example: Partial Fill Progression

Reserve: Main Market USDC 6.0% 3m

Day	Event	Ticket A (first)	Ticket B (second)	available_liquidity	queued_liquidity
0	100M deposited, 100M borrowed	—	—	0M	0M

10	Ticket A submitted for 60M	60M pending	—	0M	60M
15	Ticket B submitted for 40M	60M pending	40M pending	0M	100M
20	Borrower repays 25M	35M pending	40M pending	0M	75M
25	Borrower repays 35M	Filled, exits	40M pending	0M	40M
30	Borrower repays 40M	—	Filled, exits	0M	0M

Ticket A must fully complete before Ticket B receives anything.

6. Rollover Mechanism

Each loan in a fixed-rate reserve has a maturity time. At maturity, borrowers must either repay or extend the loan. The rollover mechanism governs how loans can be extended and under what conditions.

Maturity Calculation

When a loan originates, its maturity is set: `maturity_time = loan_start_time + duration`

Example: A loan starts Day 5 on Main Market USDC 5.5% 3m. Maturity = Day 5 + 90 days = Day 95. When `current_time >= maturity_time`, the loan reaches maturity.

Rollover Configuration

Borrowers specify rollover preferences when originating the loan. Options include:

Parameter (in order of precedence)

Behaviour at Maturity

No rollover (boolean)	Loan must be repaid by maturity or position becomes liquidatable
Attempt Same reserve (boolean)	Attempt to roll into the same fixed-rate reserve
Attempt Better rate (boolean)	Attempt to roll into a lower-rate fixed reserve if available, otherwise same reserve
Attempt Variable fallback (boolean)	Attempt fixed-rate rollover first, fall back to variable reserve if liquidity available

Rollover can only occur within the same lending market. A loan in Main Market cannot roll into JLP Market.

Borrower Choices at Maturity

- **Repay:** Pay `total_debt` (principal + accrued interest). Debt clears from the reserve. Collateral unlocks proportionally if no other loans exist on the obligation.
- **Rollover:** Extend the loan for another full term. If rolling into the same reserve, the rate stays the same. If rolling into a different reserve, the new reserve's rate applies.
- **Default:** Neither repay nor rollover within the grace period. Position becomes liquidatable regardless of health factor.

Rollover Eligibility

Rollover is not guaranteed. It depends on available liquidity and withdrawal queue state.

Term	Meaning
<code>borrowed_amount</code>	Principal of the maturing loan that wants to roll over
<code>available_liquidity</code>	Unborrowed capital currently sitting in the reserve
<code>queued_liquidity</code>	Total amount lenders have requested to withdraw

Rollover succeeds only if `borrowed_amount <= (available_liquidity - queued_liquidity)`

The principle: **lenders who want out take priority over borrowers who want to stay in.**

In practice, at least one of `available_liquidity` or `queued_liquidity` is always zero—when available liquidity exists and a withdrawal ticket is submitted, the queue immediately processes and the ticket fills. The two values don't coexist because one consumes the other. This means if any queued liquidity exists, rollover is blocked.

Grace Period

After maturity, the borrower has a grace period (e.g., 6 hours) to take action:

- Repay the loan (repayment is allowed at any point during the loan's lifespan, not just at maturity)
- Rollover into the same reserve (if eligible)
- Refinance into a different reserve within the same market

If the grace period expires with no action, the position becomes liquidatable regardless of collateral health. This ensures capital cannot remain locked indefinitely beyond the agreed term.

Example: Successful Rollover

Reserve: Main Market USDC 5.5% 3m

- Loan: 100M USDC, matures Day 90
- `available_liquidity` after maturity: 150M
- `queued_liquidity`: 20M

Rollover check: $100M \leq (150M - 20M) \rightarrow 100M \leq 130M \rightarrow \text{TRUE}$

Rollover succeeds. Loan extends to Day 180.

Examples

Successful Rollover

Reserve: Main Market USDC 5.5% 3m

Parameter	Value
Loan principal	100M USDC
Maturity	Day 90
<code>available_liquidity</code>	150M

queued_liquidity 0M

Rollover check: $100M \leq 150M - 0M \rightarrow 100M \leq 150M \rightarrow \text{TRUE}$

Rollover succeeds. New maturity extends to Day 180.

Blocked Rollover (Withdrawal Queue)

Reserve: Main Market USDC 6.0% 3m

Parameter	Value
Loan principal	80M USDC
Maturity	Day 90
available_liquidity	0M
queued_liquidity	80M (vault submitted withdrawal ticket)

Rollover check: $80M \leq 0M - 80M \rightarrow 80M \leq -80M \rightarrow \text{FALSE}$

Rollover blocked. Borrower must repay within grace period. When borrower repays, the 80M returns to the reserve as available liquidity, the withdrawal queue processes, and the vault receives its 80M.

Rollover to Different Reserve

Reserve: Main Market USDC 5.5% 3m (origin) $\rightarrow$ Main Market USDC 5.0% 3m (target)

Parameter	Value
Loan principal	50M USDC
Maturity	Day 90
Target reserve available_liquidity	60M
Target reserve queued_liquidity	0M

Borrower configured "better rate" rollover preference. At maturity, system checks 5.0% 3m reserve first.

Rollover check on target: $50M \leq 60M - 0M \rightarrow \text{TRUE}$

Rollover succeeds into lower-rate reserve. Borrower now pays 5.0% instead of 5.5% for the next term.

Summary

Fixed Rates creates interest rate certainty and price discovery through:

1. **Markets with multiple reserves:** Isolated lending environments where each debt token has variable-rate and multiple fixed-rate reserves with different terms, each with independent liquidity and maturity schedules
2. **Vaults as liquidity aggregators:** Existing infrastructure that manages lender capital across reserves, absorbing complexity of allocation and rebalancing on behalf of individual lenders
3. **Conditional Liquidity (NEW):** Allows vaults to signal fixed-rate availability with zero opportunity cost while keeping capital in variable reserves earning yields, making professional fixed-rate risk management viable at scale
4. **Borrow Orders:** Enable both (a) instant atomic activation of Conditional Liquidity when matches occur, and (b) visible demand signaling when liquidity doesn't exist yet, allowing vaults to see where borrowers want terms and potentially allocate accordingly
5. **Withdrawal queue:** Per-reserve FIFO queues with advance notice requirement, allowing vaults to manage maturity schedules and prevent indefinite capital lock while avoiding last-minute grieving
6. **Rollover mechanism:** Per-reserve checks with `queued_liquidity`, balancing borrower convenience (ability to extend loans) against vault exit rights (ability to block rollover and exit)

The system creates yield curves within each market showing term premium structure (rate differences across durations).

Vaults handle all complexity: individual lenders deposit once and receive blended returns from both variable and fixed-rate positions. Vault managers balance variable exposure (for instant lender liquidity), conditional placements (for yield enhancement via price discovery), and committed fixed positions (for locked premium yields), while managing maturity mismatches and lender withdrawal demand.

The innovation of Conditional Liquidity removes the opportunity cost and fragmentation that previously made fixed-rate lending impractical, enabling professional risk managers to provide

rate certainty to borrowers and duration-matched yields to lenders without sacrificing capital efficiency.